

Git в DevOps

Правдин Иван

24.05.2025

Какую проблему решает Git в DevOps?

Без системы контроля версий

- Код только на компьютере разработчика
- Нет истории изменений
- Нет координации в команде

С Git

- Центральное хранилище кода
- Полная история изменений
- Совместная работа команды

Git — это основа для всех DevOps практик

Git репозиторий содержит не только код:

- Исходный код приложения
- Конфигурацию инфраструктуры
- CI/CD пайплайны
- Документацию

Принцип: всё что влияет на продукт должно быть в Git

DevOps процессы запускаются от Git событий:

- Push в ветку → автоматическое тестирование
- Pull Request → код-ревью и проверки
- Создание тега → релиз новой версии
- Merge в main → деплой на прод

Git становится пультом управления для всех процессов

Зачем изучать Git в контексте DevOps?

1. Git — основа любой CI/CD системы
2. Понимание Git workflow критично для DevOps
3. Git hooks и webhooks — инструменты автоматизации

1. Git workflow в DevOps командах
2. Ветвление и стратегии слияния
3. Git hooks для автоматизации
4. GitHub/GitLab как DevOps платформа
5. Webhooks для интеграций
6. Infrastructure as Code в Git
7. Практика: настройка автоматизации через Git

Git Flow

- Сложная модель веток
- develop, release, hotfix ветки
- Подходит для больших проектов
- Медленная доставка изменений

GitHub Flow

- Простая модель: main + feature ветки
- Быстрая доставка в продакшн
- Идеально для DevOps
- Требуется хорошие тесты и CI/CD

В DevOps предпочитают GitHub Flow

Проблема: как деплоить незавершённые функции?

Решение: Feature Flags

- Код новой функции в main ветке, но отключен
- Включается через конфигурацию
- Позволяет деплоить часто без риска

Git + Feature Flags = непрерывная доставка

Монорепозиторий vs Мультирепозиторий

Монорепозиторий

- Весь код в одном репозитории
- Проще координация изменений
- Единые CI/CD пайплайны
- Используют Google, Facebook

Мультирепозиторий

- Каждый сервис в своём репозитории
- Независимые релизы
- Изоляция команд
- Больше настроек CI/CD

Выбор зависит от размера команды и архитектуры

Server-side hooks:

- pre-receive — проверка перед принятием push
- post-receive — действия после push
- post-update — обновление веб-интерфейса

Применение в DevOps:

- Запуск тестов при push
- Автоматический деплой
- Отправка уведомлений в чат

Webhook = HTTP запрос при Git событии

Типичные сценарии:

- Push → запуск Jenkins job
- Pull Request → уведомление в Slack
- Tag создан → релиз в производство
- Issue создана → задача в Jira

GitHub, GitLab, Bitbucket поддерживают webhooks

YAML файлы в `.github/workflows/`

Автоматически выполняют:

- Сборку и тестирование кода
- Линтинг и проверки безопасности
- Деплой в различные окружения
- Создание релизов

Git событие → GitHub Action → результат

Принцип: инфраструктура описывается кодом и хранится в Git

Инструменты:

- Terraform — создание инфраструктуры
- Ansible — настройка серверов
- Kubernetes YAML — оркестрация контейнеров
- Docker Compose — локальная разработка

Изменение инфраструктуры = Pull Request

GitOps = Infrastructure as Code + автоматизация

Принципы GitOps:

- Декларативное описание системы в Git
- Git как единственный источник истины
- Автоматическая синхронизация с кластером
- Откат изменений через Git revert

Инструменты: ArgoCD, Flux, Jenkins X

- Git — центральный элемент DevOps процессов
- Простые workflow лучше сложных в DevOps
- Webhooks и hooks — основа автоматизации
- Infrastructure as Code должен быть в Git
- GitOps — эволюция DevOps практик

Вопросы?